

Join Kata Answer

Noa Fingerman

1. Complete the following query so you will get Italian dishes. We would like to create a dish for each combination of pasta and sauce. Complete the query so you will get 12 dishes.

```
with Pasta as (  
  select "Fettuccine" as pasta_type  
  union select "Penne"  
  union select "Spaghetti"  
  union select "Rigatoni"),  
Sauce as (  
  select "Tomatoes" as sauce_type  
  union select "Pesto"  
  union select "Alfredo")
```

```
select *  
from Pasta  
יהיה cross join Sauce;
```

After which famous mathematician the operation that you used is named?

The mathematician is Rene Descartes

2. Complete the following query so you will get dishes and their ingredients. What should you do to have an existing ingredient that will not be part of the results?

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Pesto Penne"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine"  
  union select 2, "Pesto"  
  union select 2, "Penne"  
  union select 3, "Truffle" #ingredient without a dish (will be excluded by inner join)  
)  
select d.dish_id, d.dish_name, i.ingredient_name  
from Dishes as d  
join Ingredients as i  
on d.dish_id = i.dish_id;
```

Give the ingredient a dish_id that doesn't exist in Dishes.

3. Complete the following query so you will get dishes and their ingredients. This time you should also get dishes with no ingredients. Make sure that you will give a funny name to the ingredients lacking dishes.

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Air Soup"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine")  
select d.dish_id, d.dish_name, i.ingredient_name  
from Dishes as d  
left join Ingredients as i  
on d.dish_id = i.dish_id;
```

4. Complete the following query so you will get dishes and their ingredients. This time you should also get only dishes with no ingredients. Make sure that you will give a funny name to the ingredients lacking dishes.

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Air Soup"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine")  
  
select d.dish_id, d.dish_name,  
'Mystery Meat' as ingredient_name  
from Dishes as d  
left join Ingredients as i  
on d.dish_id = i.dish_id  
where i.dish_id is null;
```

5. Complete the following query so you will get dishes and their ingredients, just as in question 2. Put the tables in the opposite order that you did in question 2 yet get exactly the same result. What were the differences?

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Air Soup"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine")  
  
select d.dish_id, d.dish_name, i.ingredient_name  
from ingredients as i  
right join dishes as d  
on i.dish_id = d.dish_id;
```

To get the same output as question 3 after swapping the table order we use a right join instead of a left join. This ensures all records from dishes are kept even though it is now the right hand table.

6. Complete the following query so you will get dishes and their ingredients. This time you should also get only dishes with tomatoes. You can filter rows in two places. What is the difference?

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Pesto Penne"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine"  
  union select 2, "Pesto"  
  union select 2, "Penne")
```

```
select *  
from Dishes as d  
join Ingredients as i  
on d.dish_id = i.dish_id  
where i.ingredient_name = 'Tomatoes';
```

```
select *  
from Dishes as d  
join Ingredients as i  
on d.dish_id = i.dish_id  
and i.ingredient_name = 'Tomatoes';
```

Both give the same results since inner join treats on & where conditions the same way.

7. Complete the following query so you will get dishes and their ingredients. This time you should also get only dishes without tomatoes. In principle, you can filter rows in two places. What is the difference? What is the difference from the previous question?

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Pesto Penne"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine"  
  union select 2, "Pesto"  
  union select 2, "Penne")
```

```
select d.dish_id, d.dish_name  
from Dishes as d  
left join Ingredients as i  
on d.dish_id = i.dish_id  
and i.ingredient_name = 'Tomatoes'  
where i.dish_id is null;
```

If we use a simple where ingredient_name != 'Tomatoes' on an inner join, dishes that have tomatoes will still show up because of their other ingredients (like Fettuccine). Filtering inside the on part of a left join (and i.ingredient_name = 'Tomatoes') matches only the tomato rows. Dishes that do not have tomatoes produce null, and where i.dish_id is null filters out any dish that contains tomatoes while keeping only the ones that never had them.

8. Complete the following query so you will get dishes, their ingredients, and their cost.

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Pesto Penne"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine"  
  union select 2, "Pesto"),  
Ingredients_cost as (  
  select "Tomatoes" as ingredient_name, 5 as ingredient_cost  
  union select "Fettuccine", 3  
  union select "Pesto", 8)  
  
select *  
from Dishes as d  
join Ingredients as i  
on d.dish_id = i.dish_id  
join Ingredients_cost as ic  
on i.ingredient_name = ic.ingredient_name;
```

9. Complete the following query so you will get dishes, their ingredients, and their cost. This time select only fancy dishes that include an ingredient that cost at least 100.

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Truffle Pappardelle"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine"  
  union select 2, "Truffle"  
  union select 2, "Pappardelle"),  
Ingredients_Cost as (  
  select "Tomatoes" as ingredient_name, 5 as ingredient_cost  
  union select "Fettuccine", 10  
  union select "Truffle", 150  
  union select "Pappardelle", 12)
```

```
select d.dish_id, d.dish_name,  
i.ingredient_name, ic.ingredient_cost  
from dishes as d  
join ingredients as i  
on d.dish_id = i.dish_id  
join ingredients_cost as ic  
on i.ingredient_name = ic.ingredient_name  
where d.dish_id in (  
  select ing.dish_id  
  from ingredients as ing  
  join ingredients_cost as c  
  on ing.ingredient_name = c.ingredient_name  
  where c.ingredient_cost >= 100  
);
```

10. Complete the following query so you will get dishes, their ingredients, and their cost. This time select no fancy dishes that do not include an ingredient that cost at least 100. Beware here and think how to use the answer to the previous question. This question uses subqueries that appear later than join in the course. Come back here after learning them. What is the difference between this question and the previous one?

```
with Dishes as (  
  select 1 as dish_id, "Tomatoes Fettuccine" as dish_name  
  union select 2, "Truffle Pappardelle"),  
Ingredients as (  
  select 1 as dish_id, "Tomatoes" as ingredient_name  
  union select 1, "Fettuccine"  
  union select 2, "Truffle"  
  union select 2, "Pappardelle"),  
Ingredients_Cost as (  
  select "Tomatoes" as ingredient_name, 5 as ingredient_cost  
  union select "Fettuccine", 10  
  union select "Truffle", 150  
  union select "Pappardelle", 12),  
Fancy_Dishes as (  
  select distinct i.dish_id  
  from ingredients as i  
  join ingredients_cost as ic  
  on i.ingredient_name = ic.ingredient_name  
  where ic.ingredient_cost >= 100)
```

```
select d.dish_id, d.dish_name,  
i.ingredient_name, ic.ingredient_cost  
from dishes as d  
join ingredients as i  
on d.dish_id = i.dish_id  
join ingredients_cost as ic  
on i.ingredient_name = ic.ingredient_name  
left join Fancy_Dishes as f  
on d.dish_id = f.dish_id  
where f.dish_id is null;
```

The difference is that the previous question checks for existence using a join and a filter. this question checks for absence which requires a subquery to identify all dishes that don't meet the condition being asked. Using not in in the previous code is risky because if the subquery returns even a single null value, the entire condition evaluates to unknown, causing the query to return zero rows. Therefore, using an anti-join safely handles null values and is easier for the database optimizer to execute efficiently.